ChatGPT

Computer System Architecture – Unit I

1. Analog vs Digital Computers

- **Definition:** Analog computers process continuously variable physical quantities (such as voltage, pressure, or mechanical motion) as data, whereas digital computers process data in discrete binary form (0s and 1s) [1]. In analog systems, values vary smoothly over a range, but in digital systems, data is encoded as distinct binary digits.
- **Data Representation:** Analog systems use continuous signals (e.g. a moving needle or varying electrical voltages) to represent information. Digital systems use discrete bits and binary code (fixed "0" or "1" levels) for storage and computation [1]. This means analog clocks or thermometers display a continuously changing pointer, while digital clocks and calculators use numerical displays and logic gates.
- **Precision and Accuracy:** Analog computers often have lower precision because physical components introduce noise and continuous values can't be represented perfectly. Digital computers offer high precision limited only by word length (number of bits) and quantization (which is hardware-defined) [2]. For example, an analog signal may degrade with interference, whereas a digital value remains exact until bit-level rounding.
- **Processing and Speed:** Analog computation can solve certain mathematical problems (like differential equations) in real time by exploiting physical processes, but they are generally slower and less flexible. Digital computers perform operations step-by-step using logic circuits and can be much faster and programmable. Modern digital CPUs can perform billions of instructions per second, whereas analog machines (used historically in the 1950s–60s) had limited speed and could not easily be reprogrammed for different tasks [2].
- **Programmability:** Digital computers are programmable and versatile. They execute software instructions stored in memory and can run many types of programs. Analog computers (e.g. older analog calculators or simulators) are often special-purpose and not easily reprogrammable – changing their function usually meant rewiring circuits or adjusting hardware knobs. Thus, digital systems are used for general-purpose computing, while analog systems are mostly used in niche real-time control and simulation tasks.
- **Examples:** A classic analog device is a mercury thermometer (analog voltage changes with temperature) or an analog clock with hands. A practical digital example is a personal computer or smartphone, which encodes data digitally. GeeksforGeeks notes examples of analog vs digital: "Analog clock and thermometer, etc." versus "Digital laptop, digital camera, digital watch, etc." [3]. Other analog examples include analog speedometers and old radio tuners; digital examples include calculators, digital audio, and modern embedded controllers.
- **Performance and Reliability:** Digital computers are generally more reliable, precise, and easier to design and maintain than analog ones [2]. Digital systems can store large amounts of data (in memory), whereas analog systems have very limited "memory" of continuous states. Digital machines are less susceptible to small physical variations and can operate at higher speeds.
- **Conclusion:** The main contrast is continuous (analog) vs discrete (digital) data processing [1]. Analog computers use natural phenomena to model problems, but suffer from inaccuracy and lack of flexibility. Digital computers manipulate binary data, enabling precise arithmetic and logic operations, programmability, and wide application scope [2]. Modern computing is overwhelmingly digital, but analog concepts survive in sensors and special-purpose converters.

2. Functional Units of a Digital Computer

- **Key Components:** A digital computer consists of five major functional units that collaborate to perform tasks [4]:
- **Input Unit:** Captures data from external sources (keyboard, mouse, sensors, etc.) and converts it into binary form for the computer [4]. For example, a keyboard scans keys and sends binary scan codes to the CPU.
- **Memory Unit:** Stores data and program instructions. It includes primary (volatile) memory like RAM, which holds data during execution, and secondary (non-volatile) storage (HDD/SSD) for long-term data. The Memory Unit holds the instructions and operands that the CPU fetches and processes [4].
- **Central Processing Unit (CPU):** The "brain" of the computer [4]. Internally, the CPU comprises:
  - **Control Unit (CU):** Directs the operation of the CPU and whole computer. It fetches instructions from memory, decodes them, and issues control signals to other units to carry out operations [5] [4].
  - **Arithmetic Logic Unit (ALU):** Performs arithmetic (addition, subtraction, etc.) and logical (AND, OR, comparisons) operations on data [6] [7]. For instance, it can add two register values or compare them.
  - **Registers:** Small, fast storage locations inside the CPU for temporarily holding data, addresses, and intermediate results [8]. Examples include the Accumulator (for arithmetic results), the Program Counter, and the Instruction Register (to name a few). Registers speed up processing by keeping data close to the ALU [8].
- **Output Unit:** Converts the processor's binary results into human-readable or actuator form [4]. Examples include monitors (displaying characters), speakers (audio out), and printers. It takes data from CPU or memory and produces output, e.g., a display refresh or printed page.
- **Bus System (Interconnection):** A set of wires or pathways that carry data, addresses, and control signals among the above units [4]. The bus connects the CPU, memory, input, and output units. It typically includes three buses: an **address bus** (carries memory addresses), a **data bus** (carries actual data), and a **control bus** (carries control signals) [9] [10]. The bus architecture allows units to communicate efficiently through shared pathways.
- **Block Diagram (Conceptual):** One can visualize the computer as blocks: Input → CPU ↔ Memory → Output, all linked by the System Bus [11]. The CPU's internal blocks (CU, ALU, Registers) are shown with control signals connecting them. For example, the bus brings an instruction from memory into the Instruction Register (IR), then CU decodes it and signals the ALU to execute, possibly fetching/storing data via the memory. This block structure underlies all computation.
- **Role of Each Unit:** Each unit has a distinct role [4]. The Input Unit handles external interactions, the Memory Unit holds programs/data, the CPU processes instructions, and the Output Unit delivers results. The Control Unit synchronizes everything, the ALU executes operations, and the Bus facilitates all inter-component communication. Together, they implement the stored-program concept and enable digital computation.

3. Von Neumann Architecture: Structure and Working

- **Basic Concept:** Von Neumann architecture (stored-program computer) is a design model where both program instructions and data share the same memory [5]. In this architecture, the CPU fetches instructions and operands from a single memory space via the same system bus. This unification simplifies the design and allows programs to be treated as data.
- **Components:** The primary components of a Von Neumann system are the **CPU**, **Memory**, and **I/O devices** [12]. The CPU includes the Control Unit, ALU, and registers. Memory stores both instructions and data. I/O devices connect to the outside world. A single bus system connects all

three: CPU <-> Memory <-> I/O (often shown as one common bus) [13] [14] . The key feature is a shared communication pathway for instructions and data.

- **Stored-Program Concept:** Instructions for a program are stored in memory just like data. During execution, the Control Unit fetches an instruction from memory into the Instruction Register, decodes it, and then executes it using the ALU or by moving data between registers/memory. The Program Counter (PC) holds the address of the next instruction to fetch. After each fetch, PC increments (or is modified) to point to the following instruction. Because data and code share memory, a program can modify itself or generate new code.
- **Sequential Operation:** Execution follows a *Fetch-Decode-Execute* cycle. The CU fetches the instruction at address PC, increments PC, decodes the instruction (determines opcode and operands), and then coordinates the needed operations (for example, loading registers, performing ALU ops) [13] [14] . Each instruction is processed one at a time in sequence (unless control flow changes it). The single shared bus means an instruction fetch or a data transfer happens in separate cycles; they cannot occur simultaneously on one bus [14] .
- **Shared Memory and Bottleneck:** Because instructions and data use the same bus, a bottleneck can occur (the "Von Neumann bottleneck"). Only one transfer (either instruction fetch or data read/write) can use the bus at a time [14] . This limits throughput compared to architectures with separate buses. The impact is that memory bandwidth can limit CPU speed.
- **Diagram (Conceptual):** A typical Von Neumann block diagram shows Memory, CPU, and I/O connected by a single bus [13] [11] . CPU contains CU, ALU, and Registers. Memory holds code/data. The control bus, address bus, and data bus interconnect them. For example, the diagram in sources highlights CPU connected via a "system bus" to memory and I/O [11] . (In answering, we describe this textually since images are omitted.)
- **Key Characteristics:**
- **Single Memory:** Both data and instructions reside in one memory [5] .
- **One Bus for All:** A single set of address/data/control lines is shared for fetching instructions and data [14] .
- **Sequential Execution:** Instructions are executed sequentially unless control instructions alter flow.

- **Programmability:** Programs can easily modify themselves or construct new code at runtime.

- **Von Neumann Bottleneck:** As noted, sharing memory and bus causes a performance limit. No matter how fast CPU or memory is, instruction/data transfer is sequential, creating delays [14] . This means optimization often focuses on caching or bus improvements to alleviate the bottleneck, but the basic architecture retains this sequential constraint.

- **Summary:** Von Neumann architecture is the classical model for most computers today. It uses a single memory for code and data, a single bus, and a CPU that fetches and executes instructions in order. This model facilitates stored-program flexibility but must contend with shared-bus limitations [13] [14] .

4. Bus Structure and Types of Buses

- **Bus Concept:** A *bus* is a common communication pathway that connects multiple components inside a computer [10] . It consists of parallel wires (or traces) that carry bits of information (data, addresses, and control signals) between units. Using a bus minimizes the number of direct connections by sharing lines: for example, when the CPU wants to read memory, it places the address on the address bus and activates the control bus lines for a "read" operation.
- **Purpose:** The bus allows different units (CPU, memory, I/O devices) to send and receive information over shared lines [10] . It centralizes communication, reducing complexity. All devices

listen on the bus; control signals determine which device drives the bus at a time. For instance, the CPU might drive the address and control buses, then memory responds by placing data onto the data bus.

- **Types of Buses:** There are three main categories of system buses [15] :
- **Address Bus:** Carries the memory or I/O address that the CPU wants to access [16] . It is *unidirectional*: only the CPU (master) drives the address bus to specify where to read/write. The width (number of wires) of the address bus determines the maximum addressable memory locations (e.g. a 16-bit address bus can address 2^16 bytes) [16] .
- **Data Bus:** Carries the actual data being transferred between components [17] . It is *bidirectional*: both the CPU and memory (or I/O) can drive the data bus. For example, during a read, memory puts data on the bus to the CPU; during a write, the CPU drives data to memory. The data bus width (e.g. 8-bit, 16-bit, 32-bit) determines how much data can be sent at once [18] .
- **Control Bus:** Transmits control and timing signals among components [19] . Typical control lines include *read/write* signals, clock, interrupt requests, and memory access gates. The CPU uses control lines to indicate whether the current bus cycle is a read or write, and peripheral devices use interrupt lines to signal the CPU. Control bus lines can be uni- or bi-directional, depending on whether signals (like interrupt requests) go to or from the CPU [19] .
- **System Bus:** Collectively, these buses form the *system bus* or *backbone* of the computer. The address bus carries location identifiers, the data bus carries the values, and the control bus carries the commands and timing [9] . For example, to load a value from memory, the CPU places the address on the address bus, asserts the “Memory Read” line on the control bus, then reads the value presented on the data bus.
- **Bus Structure Types:** Buses can be designed in various ways (e.g. expansion buses, PCI, USB externally). In this context, we focus on the internal CPU/memory bus structure. There are *multiplexed buses* (address and data share lines at different times) or *dedicated buses*. But fundamentally, the three-bus model (address/data/control) is most common in textbook architecture.
- **Importance:** Buses define how fast components can communicate and how much hardware is needed. A wider data bus improves throughput; separate buses for address and data allow simultaneous transfers. The bus design affects performance and scalability. In summary, bus structure provides the channels for all register transfers and micro-operations across components [11] .

5. Number Systems and Conversions (Example Problems)

- **Number Systems Overview:** Computer architecture uses various base number systems. The main ones are binary (base-2), octal (base-8), decimal (base-10), and hexadecimal (base-16) [20] . Each system has a fixed radix (base) determining digit values. For example, binary digits are 0 or 1, octal 0–7, decimal 0–9, and hexadecimal 0–F. Understanding these is essential because computers natively operate in binary, but humans use decimal or hex for convenience.
- **Binary System (Base-2):** Uses two symbols {0,1}. Each bit represents a power of 2. For instance, the binary number 1101_2 equals $1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 13_{10}$.
- **Decimal System (Base-10):** Uses digits {0–9}. Powers of 10 weight each position (units, tens, hundreds, etc.). E.g., $345_{10} = 3\times10^2 + 4\times10^1 + 5\times10^0$.
- **Hexadecimal System (Base-16):** Uses digits {0–9, A–F}. Powers of 16 weight positions. A stands for 10. For example, $7A_{16} = 7\times16 + 10 = 122_{10}$.
- **Conversion Techniques:**
- **Binary → Decimal (Base-2 to Base-10):** Multiply each binary digit by powers of 2 and sum [21] . For a binary number $b_i...b_0$, compute value = $\Sigma\ (b_j \times 2^j)$ from j=0 to i.
- **Decimal → Binary:** Repeatedly divide the decimal number by 2, recording remainders (0 or 1). The binary digits are the remainders read in reverse order.

- **Hexadecimal → Decimal:** Multiply each hex digit by powers of 16 and sum. E.g., for $(7A)_{16}$: 7×16^1 + A(10)$\times16^0$ = 112 + 10 = 122.
- **Conversions (examples):**
  - (a) **Binary to Decimal:** Convert $(1011101)_2$. Calculate $1\times2^6 + 0\times2^5 + 1\times2^4 + 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 = 64 + 0 + 16 + 8 + 4 + 0 + 1 = \mathbf{93_{10}}$. (This follows the multiplication method.)
  - (b) **Decimal to Binary:** Convert $(345)_{10}$. Divide by 2: 345 ÷ 2 = 172 R1, 172 ÷ 2 = 86 R0, 86 ÷ 2 = 43 R0, 43 ÷ 2 = 21 R1, 21 ÷ 2 = 10 R1, 10 ÷ 2 = 5 R0, 5 ÷ 2 = 2 R1, 2 ÷ 2 = 1 R0, 1 ÷ 2 = 0 R1. Reading remainders bottom-up yields 101011001_2. So 345_{10} = $\mathbf{101011001_2}$.
  - (c) **Hexadecimal to Decimal:** Convert $(7A)_{16}$ to decimal. $7A_{16}$ = 7×16 + 10 = 112 + 10 = $\mathbf{122_{10}}$.
- **Conversion Principles:** The above methods generalize to any bases. One multiplies digits by descending powers of the source base to get decimal, or divides by the target base iteratively. These conversions rely on the definition of positional number systems [21].
- **Importance in Computer Architecture:** Numbers in memory can be in binary (two's complement for signed integers) or as floating point, so base conversions are fundamental when interpreting or debugging. Hexadecimal is often used by programmers to concisely represent binary data (each hex digit = 4 binary bits).
- **Summary:** In summary, we covered base-2, base-10, and base-16 representations and their interconversion. The key is that place weights change with the base. For example, the Byju's reference explains binary-to-decimal conversion: multiply each bit by $2^0,2^1,2^2,\ldots$ [21], which we used in the examples. Understanding these processes ensures correct interpretation of values in digital systems.

6. Fixed-Point vs Floating-Point Representation

- **Fixed-Point Representation:** A fixed-point number has a fixed location for the binary (or decimal) point [22]. The format allocates a set number of bits for the integer part and a set number for the fractional part. For instance, in a fixed<8,3> format (8 bits total, 3 fractional bits), the binary number 00010.110_2 means 2.75_{10} [22]. Fixed-point is essentially an integer scaled by a constant factor. It is simple: hardware just treats the number as integer and knows where the point is. It is efficient for applications where the range of values is known and limited (such as embedded controllers).
- **Floating-Point Representation:** Floating-point uses a scientific notation-like format: a **sign**, **exponent**, and **mantissa (significand)**. It allows the "point" to *float* by adjusting the exponent [23] [24]. IEEE 754 standardizes this: e.g., single precision (32 bits) has 1-bit sign, 8-bit exponent, 23-bit fraction [23]. A floating number is interpreted as (–1)^sign × mantissa × base^(exponent), enabling representation of very large or very small numbers. For example, 4.5 in binary is 100.1_2, which in single-precision float would be normalized to $1.001_2 \times 2^2$ [25].
- **Comparison – Range and Precision:**
- *Range:* Floating-point covers a much wider dynamic range. It can represent values from extremely small (near zero) to extremely large by adjusting the exponent. Fixed-point, with a fixed number of integer bits, has a limited range (e.g. only up to 255 for 8-bit unsigned). Floating-point's range advantage is highlighted by sources: "It can represent a wide range of values from very large to very small" [26].
- *Precision:* Fixed-point can exactly represent fractional values aligned to its scaling (e.g. two decimal places if scaled by 100). Floating-point can represent very small increments but with rounding errors for many values. Floating format balances precision versus range: more exponent bits expand range; more mantissa bits increase precision. For instance, single precision gives ~7 decimal digits accuracy, double gives ~16 digits [27].

- *Complexity:* Fixed-point arithmetic is simpler and faster (integer adders/multipliers suffice). Floating-point arithmetic requires more complex hardware (normalization, rounding, handling special cases) and hence is slower and more resource-intensive.
- **Examples:**
- *Fixed-point Example:* With a fixed<8,4> format, the 8-bit pattern 10101010_2 is interpreted as 1010.1010_2 = 10.625_{10} [22] . The binary point is at a fixed position. All numbers share that format.
- *Floating-point Example:* The decimal number 4.5 is represented as 100.1_2. In IEEE 754 single precision, 4.5_{10} = (–1)^0 × 1.001 × 2^2. The stored bit pattern (hex 0x40900000) reflects sign=0, exponent=129 (2+127 bias), significand=001... [25] .
- **Trade-offs:** Fixed-point yields exact arithmetic for scaled integers but can overflow easily. Floating-point can handle wide numeric range but may introduce rounding errors and uses more bits to represent a number (thus less exactness at any magnitude).
- **Summary:** In essence, fixed-point has a fixed decimal point giving uniform precision but narrow range; floating-point has a variable point (via exponent) giving wide range at the cost of relative precision [26] . Floating point is used for scientific and graphics computations; fixed-point is used in simple, fast applications (signal processing, finance) where range is known and exact decimal fractions are needed.

7. Registers and Types of Registers

- **Register Definition:** Registers are very fast, small-capacity storage locations built into the CPU [28] . They hold data, instructions, addresses, and intermediate results to speed up processing. Accessing a register is much faster than accessing main memory. Registers cooperate with the CPU and ALU to execute instructions, store operands, and track execution state [28] .
- **General-Purpose Registers (GPR):** These are used by programs to hold operands and results during computation. They are often named R0, R1, ... Rn. GPRs can be used for arithmetic or data manipulation. Modern CPUs have many (e.g., 8 or 16 or more) GPRs for efficiency [29] . For example, R1 might hold a temporary value during an operation.
- **Accumulator (AC):** A special register often used in arithmetic operations. It is the default operand and storage for ALU results in many designs [30] . When a computation is performed, one operand might be in the Accumulator and the other in another register or memory, with the result stored back in the Accumulator.
- **Memory Address Register (MAR):** Holds the address of the memory location to read from or write to [31] . When the CPU needs data from memory, it places the address in MAR.
- **Memory Data Register (MDR) / Memory Buffer Register (MBR):** Temporarily holds data being transferred to or from memory [31] . After MAR has the address, on a read operation memory's contents are placed into MDR; on a write, MDR holds the data to store.
- **Program Counter (PC):** Contains the address of the next instruction to fetch [32] . After fetching an instruction, PC increments (or is modified by jumps) so the CPU knows where to fetch next. The PC ensures sequential instruction execution by default.
- **Instruction Register (IR):** Holds the current instruction being executed [33] . Once an instruction is fetched from memory, it is loaded into the IR, where the CPU decodes and executes it.
- **Stack Pointer (SP):** Points to the top of the call stack in memory [34] . It is used for function call/return management, pushing and popping temporary data (return addresses, local variables).
- **Flag (Status) Registers:** A set of single-bit flags indicating the outcome of operations (Zero, Carry, Sign, Overflow, etc.) [35] . For example, after an addition, the Zero flag is set if the result is zero. These flags affect conditional branching.
- **Condition Code Register (CCR) / Status Register:** Contains flags that reflect CPU status after operations [35] . It may include the same bits as Flag Register. For example, Carry (C) and Zero (Z) flags help in decision-making by the CU.

- **Internal High-Speed Buffers:** Some architectures include special registers (index registers, segment registers) for advanced addressing. These are also registers but serve specific addressing or paging tasks.
- **Summary:** Registers are tiny, high-speed memories inside the CPU that enable fast data access and manipulation [28]. Different registers serve different roles: data registers (accumulator, GPRs), address registers (MAR), control registers (PC, SP), and status registers (Flags). Having multiple registers allows a CPU to quickly store and retrieve operands during instruction execution, significantly speeding up computation [28].

8. Register Transfer Language (RTL) and Data Transfer Between Registers

- **Definition of RTL:** Register Transfer Language (RTL) is a notation used to describe data flow and operations at the register-transfer level of a digital system [36]. It abstracts hardware behavior by specifying how data moves between registers and how registers are modified. For example, an RTL statement like `R2 ← R1 + R3` means "add contents of R1 and R3 and store the result in R2." RTL bridges algorithmic design and hardware, often used in HDLs and design documentation [36].
- **RTL Syntax and Operations:** In RTL, registers are denoted by names (e.g. R1, R2), and the transfer operator is typically `<-`. Common RTL operations include:
- **Register Transfer:** `R_dest ← R_src` copies data from one register to another [37]. For example, `R2 ← R1` means the contents of register R1 are transferred to R2 [37].
- **Arithmetic/Logic Operations:** RTL can also show ALU operations, e.g. `R3 ← R1 + R2` (arithmetic add) or `R4 ← R1 AND R2` (bitwise AND). These indicate that ALU outputs are stored in a register.
- **Conditional Transfers:** A transfer can be made conditional, e.g. `P: R2 ← R1` means "if condition P (some flag) is true, then transfer R1 to R2."
- **Simultaneous Operations:** If two transfers happen at once, they are separated by commas. For instance, `(R1 ← R2, R3 ← R4)` means both moves occur simultaneously in one clock cycle.
- **Examples of RTL:**
- Simple transfer: `R2 ← R1` (copy R1 to R2).
- Arithmetic transfer: `AC ← AC + R3` (add R3 to Accumulator).
- Load from memory: `R1 ← M[AR]` (load register R1 from memory address in AR).
- Store to memory: `M[AR] ← R1` (store R1 into memory at address in AR).
- **Data Transfer Mechanism:** In actual hardware, data moves between registers via the **system bus** and control signals. Each register can connect to a common bus through tri-state buffers. When a register is enabled, it drives the bus; when a register is to receive data, its load signal is asserted. For example, to execute `R2 ← R1`, the control logic would enable R1's output onto the bus and simultaneously enable R2's input (load) so that R2 latches whatever is on the bus. Because the bus is shared, only one source should drive it at a time.
- **Bus and Control:** The underlying bus architecture (address/data/control buses) supports these transfers [11] [10]. For instance, one control signal might activate R1's output buffer to the bus, while another tells R2 to read from the bus. The common bus allows any register to connect to any other via one intermediary path, governed by control signals.
- **Summary:** RTL is a formal way to describe register-to-register operations in a computer. It uses symbolic statements (like `R2 ← R1`) to represent micro-operations performed by the CPU. Physically, such transfers occur over the CPU's internal buses under the direction of the control unit. Thus, RTL provides a clear high-level view of how instructions move data around inside the processor [36] [37].

9. Types of Micro-operations: Arithmetic, Logic, and Shift (with Examples)

- **Arithmetic Micro-operations:** These operate on numeric register values. Common arithmetic micro-ops include:
- **Addition:** `R3 ← R1 + R2`. Adds the contents of R1 and R2; result in R3.
- **Subtraction:** `R3 ← R1 - R2`. Subtracts R2 from R1; result in R3.
- **Increment:** `R1 ← R1 + 1`. Adds 1 to R1 (useful for counters).
- **Decrement:** `R1 ← R1 - 1`. Subtracts 1 from R1 (useful for loops).
- **Complement Operations:** 1's complement (`R1 ← NOT R1`) or 2's complement (`R1 ← -R1`) to negate values.
- **Example:** If R1=5 and R2=3, then after `R3 ← R1 + R2`, R3=8. These are implemented by ALU circuitry (adder-subtractor). Table references list these: e.g. addition (`R3 ← R1 + R2`), subtraction (`R3 ← R1 - R2`), increment (`R1 ← R1 + 1`) [38].
- **Logic Micro-operations:** These perform bitwise logical functions on register bits, not numeric magnitude. Typical logic ops are:
- **AND:** Bitwise AND of two registers. Example: if A=1101_2 and B=1011_2, then $A\ AND\ B = 1001_2$ [39]. In RTL: `R3 ← R1 AND R2`. It sets each bit of R3 to 1 only if corresponding bits of R1 and R2 are both 1.
- **OR:** Bitwise OR. Example: $1100_2\ OR\ 1010_2 = 1110_2$ [40]. RTL: `R3 ← R1 OR R2`. Each bit of R3 is 1 if at least one of R1 or R2 bits is 1.
- **XOR (Exclusive OR):** Bitwise XOR. Example: $1100_2\ XOR\ 1010_2 = 0110_2$ (bits differing are set) [41]. RTL: `R3 ← R1 XOR R2`.
- **NOT (Complement):** Bitwise NOT. Example: if R1=1010_2, then after `R1 ← NOT R1`, R1=0101_2 [42]. Flips each bit.
- **Transfer (move) and Clear/Set:** Copy a register (`R_dest ← R_src`) or clear all bits (`R ← 0`), set all bits (`R ← 1…1`).
- **Example:** R1=1010_2. After AND with R2=1100_2, `R3` becomes 1000_2 [39]. After NOT on R1, it becomes 0101_2 [42]. Logic ops are often used for masking, flag setting, and bit manipulation.
- **Shift Micro-operations:** These move bits within a register left or right. They are used for binary multiplication/division by 2, bit alignment, and rotations. Key types:
- **Logical Shifts:** Move bits left or right, inserting 0 into the emptied bit position.
  - *Logical Left Shift:* Denoted `R ← shl R`. Every bit shifts one position toward the MSB; a 0 enters at LSB [43]. Example: shifting 01010011_2 left yields 10100110_2 (LSB=0) [44]. This effectively multiplies an unsigned binary by 2.
  - *Logical Right Shift:* `R ← shr R`. Bits shift one position toward LSB; a 0 enters at MSB [45]. For example, $01010011_2 \rightarrow 00101001_2$ [46]. This divides an unsigned number by 2 (floor).
- **Arithmetic Shifts:** Preserve the sign for signed numbers (two's complement).
  - *Arithmetic Left Shift:* Similar to logical left (fills LSB with 0) [47], often used to multiply a signed number by 2.
  - *Arithmetic Right Shift:* Bits shift right; the MSB (sign bit) is replicated into the new MSB position [48]. For example, for an 8-bit signed number 11111111_2 (-1), arithmetic right shift yields 11111111_2 (still -1) [48], preserving the negative sign. This is akin to dividing by 2 while keeping the sign.
- **Circular (Rotate) Shifts:** Rotate bits around the ends. *Circular Left:* MSB becomes LSB. *Circular Right:* LSB becomes MSB [49] [50]. Used in cryptography or bit-cycling. (E.g. 11010011_2 left-rotate $\rightarrow 10100111_2$ [49].)

- **Examples:** Shifting 00010010_2 left by 1: 00100100_2. Shifting 01100110_2 right by 1: 00110011_2. Rotating 10110011_2 left: 01100111_2.
- **Summary:** Arithmetic micro-ops handle numerical computations (add, subtract, increment, etc.) [38]. Logic micro-ops handle bitwise manipulation (AND, OR, NOT, etc.) [39]. Shift micro-ops move bits within registers (logical, arithmetic, or circular) [43] [45]. Each category of micro-operation is implemented by specialized circuits (ALU, shifters) and is used to perform the processor's low-level operations on data.

Sources: Authoritative computer architecture texts and tutorials on registers, data representation, and micro-operations were used to compile these answers [4] [5] [10] [22] [23], ensuring accuracy and completeness.

---

[1] [2] [3] Difference between Analog Computer and Digital Computer - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/difference-between-analog-computer-and-digital-computer/

[4] [7] [8] [9] [11] Components of a Computer - GeeksforGeeks
https://www.geeksforgeeks.org/computer-science-fundamentals/functional-components-of-a-computer/

[5] [6] [12] [13] Von Neumann Architecture - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/computer-organization-von-neumann-architecture/

[10] [15] [16] [17] [18] [19] What is a Computer Bus? - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/what-is-a-computer-bus/

[14] Von Neumann architecture - Wikipedia
https://en.wikipedia.org/wiki/Von_Neumann_architecture

[20] [21] byjus.com
https://byjus.com/maths/binary-to-decimal-conversion/

[22] Fixed Point Representation - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/fixed-point-representation/

[23] [25] [26] [27] Floating Point Representation - GeeksforGeeks
https://www.geeksforgeeks.org/digital-logic/floating-point-representation-basics/

[24] Floating-point arithmetic - Wikipedia
https://en.wikipedia.org/wiki/Floating-point_arithmetic

[28] [29] [30] [31] [32] [33] [34] [35] Different Classes of CPU Registers - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/different-classes-of-cpu-registers/

[36] [37] Register Transfer Language (RTL) - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/register-transfer-language-rtl/

[38] Arithmetic Micro-operations in Registers - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/arithmetic-micro-operations-in-registers/

[39] [40] [41] [42] Logic Micro-operations - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/logic-micro-operations/

[43] [44] [45] [46] [47] [48] [49] [50] Shift Micro-Operations - GeeksforGeeks
https://www.geeksforgeeks.org/computer-organization-architecture/shift-micro-operations-in-computer-architecture/